Temperature Control with stochastic dynamic programming

Duarte Antunes

A thermostat is a component of several devices (e.g. air conditioners, building heating, etc) that regulates the temperature of a given system. It exerts control by switching heating or cooling devices on or off, when the difference between a desired constant temperature and the current temperature is large. Therefore, as opposed to most control systems, where the actuation is applied at every time step, the control input is applied at a reduced number of times steps. This reduces the risk of equipment damage. Inspired by the thermostat the goal of this exercise is to establish that the optimal policy for a simple model of temperature control taking into account that control actions are expensive is to switch on the heating/cooling when the difference between the actual temperature and a constant reference exceeds a threshold.

Consider the following model for the evolution of the temperature T_k at time $k \in \{0, 1, ..., h\}$, quantized with a step ϵ,

$$T_{k+1} = T_k + U_k + w_k$$

where U_k is the temperature control input and wk are independent and identically distributed random variables. Let $\delta_k := T_k - T_{\text{des}}$ denote the offset with respect to a desired constant temperature T_{des}, and suppose that when the temperature control input is applied it sets the temperature to the desired one $U_k = T_{\text{des}} = T_k$. Then we can write

$$\delta_{k+1} = \begin{cases} \delta_k + w_k \text{ if } u_k = 0 \\ w_k \text{ if } u_k = 1 \end{cases}$$

where $u_k = 1$ if the temperature control input is applied and $u_k = 0$ otherwise; note that u_k can then be seen as new control input. Assume that initially, $T_0 = T_{des}$ which implies that $\delta_0 = 0$. We will consider a quantized model for δ_k by considering that the disturbances live in a finite set with $n_r = 2n_w + 1$ elements, where n_w is a positive integer, and are characterized by

$$\text{Prob}\left[w_k = \epsilon \bar{w}_i\right] = r_i$$

with $\bar{w}_i = -n_w + i - 1$ for $i \in \{1, \dots, n_r\}$and $\sum_{i=1}^{n_r} r_i = 1$. Then $\delta_k \in \Delta_k := \{-\epsilon k n_w, \dots, \epsilon\,(k n_w - \ 1), \epsilon k n_w\}$. Consider the following cost

$$G(h) := \left(\sum_{k=0}^{h-1} |\delta_k| + \lambda u_k \right) + |\delta_h|$$

where $|\delta_k|$ penalizes temperature deviations and λu_k term penalizes the number of temperature control inputs needed to control the system in the horizon h, where λ is a positive tunning knob expressing the trade-off between the two costs. It can be proven that if $r_{nw+1+i} = r_{nw+1-i}$ for every $i \in \{1, \dots, n_w\}$, then there exists an $L \in \Delta := \{\epsilon k | k \in \mathbb{Z}\}$, such that

$$u_k = \begin{cases} 1 \text{ if } |\delta_k| \geq L \\ 0 \text{ otherwise} \end{cases}$$

is the optimal policy that minimizes the average cost $\lim_{h\to\infty} \frac{G(h)}{h}$.

Functions `thermostatcontrol` and `asymptotictermostatcontrol` below compute the optimal finite horizon and average cost policy, respectively. You can use the following script to test them.

```
import numpy as np
```

The function `asymptotictermostatcontrol` provides the threshold of the optimal policy. The inputs are the same as in `thermostatcontrol` and the output is Lint, which is an integer such that the optimal threshold is $L = \epsilon$Lint. The rational behind his function is to compute the asymptotic probability distribution of the underlyling Markov process once the policy is fixed, i.e. for each . This can be done by computing the transition matrix of this Markov process, denoted by P and retrieving the right eigenvector associated with the unitary eigenvalue. The average cost is then an expected value of the running cost over this distribution.

```
def asymptotictermostatcontrol(r, lmbda):

    # Initialize variables

    nr = int(len(r))
    nw = int((nr - 1) / 2)

    # cost of a given L_1
    L_1 = 0
    check = 0
    cost = np.array([])

    while check == 0:
        # matrix P  ------------------------------------------------------------
        A1 = np.zeros([2 * (nw + L_1) + 1, nw])
        A1[nw + L_1 + 1 - nw - 1:nw + L_1 + 1 + nw, 1 - 1:nw] = np.matmul(np.vstack(r),
                                                                          np.ones([1, nw]))
        A2 = np.zeros([2 * (nw + L_1) + 1, 2 * L_1 + 1]);

        for x in range(0, 2 * L_1 + 1):
            A2[1 + x - 1:nr + x, x] = r

        P = np.hstack([A1, A2, A1])

        cvec = np.zeros([2 * (nw + L_1) + 1, 1])
        cvec[L_1 + nw + 1 - 1] = 0
        cvec[1 - 1:L_1 + nw] = np.vstack(np.arange((L_1 + nw), 1 - 1, -1))
        cvec[L_1 + nw + 2 - 1:] = np.vstack(np.arange(1, L_1 + nw + 1, 1))
        cvec[1 - 1:nw] = cvec[1 - 1:nw] + lmbda
        cvec[-nw + 1 - 1:] = cvec[-nw + 1 - 1:] + lmbda
        # ----------------------------------------------------------------------
        [val, vec] = np.linalg.eig(P)
        ind = np.argmin(np.abs(1 - val), axis=None)
        pss = vec[:, ind]

        cost = np.append(cost, np.matmul(pss / sum(pss), np.vstack(cvec)))

        if L_1 > 0:
            if cost[L_1 + 1 - 1] > cost[L_1 - 1]:
                L = L_1 # if the next L leads to larger cost choose previous
                break
        L_1 = L_1 + 1
```

```
    return L
```

The matlab function `thermostatcontrol`, implements the finite horizon problem and computes the optimal policy and costs to go, is deployed below, for a set of probabilities r, a penalizing coefficient lambda and a horizon of h stages. The array J contains the costs-to-go, and u is the optimal action array.

The number of stages h and the penalizing coefficient λ can be tuned to different values, depending on the problem setup.

```
def thermostatcontrol(r, lmbda, h):
    # initialize variables
    nr = int(len(r))
    nw = int((nr - 1) / 2)
    pos0 = np.zeros([1, h + 1])
    pos0[:, h] = np.array([nw * h+1])
    c = np.concatenate((np.arange(nw * h, 1 - 1, -1), np.arange(0, nw * h + 1, 1)), axis=None)

    J = [[] for _ in range(h + 1)]


    for x in range(0, 2 * nw * h + 1):
        J[h] = np.append(J[h], c[x])  # terminal cost

    u = [[] for _ in range(h)]
    check = [[] for _ in range(h)]

    # run DP
    for k in range(h - 1, -1, -1):
        pos0[:, k] = nw * k + 1
        for m in range(0, 2 * k * nw+1):

            c1 = lmbda + c[int((m+1 - pos0[:, k] + pos0[:, h])-1)]
            c0 = c[int(((m+1 - pos0[:, k]) + pos0[:, h])-1)]
            for ell in range(0, nr):
                c1 = c1 + r[ell] * J[k + 1][int((pos0[:, k + 1] + ell+1 - nw-1)-1)]
                c0 = c0 + r[ell] * J[k + 1][int((pos0[:, k + 1] + (m + 1 - pos0[:, k]) + ell+1 - nw-1)-
            J[k] = np.append(J[k], np.amin([c0, c1]))
            u_ = np.argmin([c0, c1])
            if np.abs(c0 - c1) < 1e-8:
                u[k] = np.append(u[k], 0)
            else:
                if u_ == 0:
                    u[k] = np.append(u[k], 0)
                else:
                    u[k]= np.append(u[k], 1)
    return u, J
```

```
r = np.array([0.1,0.2,0.4,0.2,0.1])
h = 100
lmbda = 10
```

```
Lint = asymptoticthermostatcontrol(r,lmbda)
```

```
[u, J] = thermostatcontrol(r, lmbda, h)
```

```
u
```

```
J
```

```
Lint
```